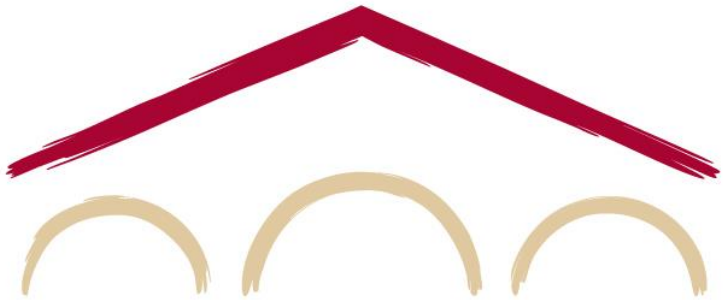
自然语言处理
深度学习
CS224N/Ling284



杨迪一

讲座 2：词向量

讲座计划

讲座 2：词向量

1. 课程组织（3分钟）
2. Word2vec 介绍（15分钟）
3. Word2vec 目标函数梯度 (25 分钟)
4. 优化基础（5分钟）
5. 我们能通过计数更有效地捕捉词义的本质吗？ (10m)
6. 评估词向量（10分钟）

关键目标： 理解单词的含义可以用高维向量表示实数，并且在课程结束时能够阅读词嵌入论文

1. 课程组织

- **审计/候补名单**
  - 如有其他问题，请发送邮件至 cs224n-win2526-staff@lists.stanford.edu
- **欢迎参加办公时间/帮助会议！**
  - 今天开始了
  - 来讨论最终项目想法以及作业
  - 尽量早点来，常来，错开时间！
- **助教办公时间：周一–周六，每次3小时，有多位助教**
  - 只需到场！我们友好的课程工作人员会在场协助您！
  - https://web.stanford.edu/class/cs224n/office_hours.html
- **教师办公时间（默认亲自）：**
  - 第一：星期二 3:30-4:30pm
  - 叶金：星期五 4:30-5:30pm

2. 我们如何表示一个词的意义?

定义： 意义（韦氏词典）

- 由一个词、短语等所表示的概念。
- 一个人通过使用词语、符号等想要表达的概念。
- 在一部作品中表达的思想，例如文字、艺术等。

常见的语言思维方式：

能指（符号）⇔ 所指（概念或事物）

= 指称语义学

树⇔ {  ,  ,  , …}

我们如何在计算机中获得可用的意义?

之前最常见的自然语言处理解决方案： 使用，例如，WordNet，一个包含同义词集 和 上位词（“是一个”关系）的词典

例如，包含“好”的同义词集合：

```
来自nltk.corpus 导入wordnet 作为wn
词性 = { 'n':'名词', 'v':'动词', 's':'形容词 (s)', 'a':'形容词', 'r':'副词'}
对于 同义词集 在wn.synsets ("好"):
    打印("{}: {}".format(词性[synset.pos()],
            ", ".join([l.name() 对于l 在synset.lemmas()])))
```

```
名词: 好
名词: 好，善良
名词: 好，善良
名词: 商品，贸易品，好
形容词: 好
形容词（饱和）: 满的，好
形容词: 好
形容词（饱和）: 可估计的，好，值得尊敬的，受人尊敬的
形容词（饱和）: 有益的，好
形容词（饱和）: 好
形容词（饱和）: 好，公正，正直
…
副词: 好，良好
副词: 彻底，健全，好
```

例如，“熊猫”的上位词：

```
来自nltk.corpus 导入wordnet 作为wn
熊猫 = wn.synset("panda.n.01")
上位 = lambda s: s.hypernyms()
列表(panda.closure(上位))
```

```
[Synset('procyonid.n.01'),
Synset('carnivore.n.01'),
Synset('placental.n.01'),
Synset('mammal.n.01'),
Synset('vertebrate.n.01'),
Synset('chordate.n.01'),
Synset('animal.n.01'),
Synset('organism.n.01'),
Synset('living_thing.n.01'),
Synset('whole.n.02'),
Synset('物体.n.01'),
Synset('物理实体.n.01'),
Synset('实体.n.01')
```

使用像 WordNet 这样的资源时的问题

- 一个有用的资源，但缺乏细微差别：
  - 例如，“熟练的”被列为“好的”的同义词
    这在某些上下文中是正确的
  - 此外，WordNet在某些同义词集中列出了冒犯性的同义词，但没有涵盖词语的内涵或适当性
- 缺失词语的新含义：
  - 例如，恶毒的、牛逼的、巧妙的、巫师、天才、忍者、最棒的
  - 无法保持最新！
- 主观的
- 需要人力来创建和调整
- 无法用于准确计算词语相似性（见后面的幻灯片）

将词语表示为离散符号

在传统的自然语言处理（NLP）中，我们将词视为离散符号：

酒店, 会议, 汽车旅馆– 一种地方主义表示

意味着一个1，其余为0

这些词的符号可以用 一热 向量表示：

汽车旅馆 = [0 0 0 0 0 0 0 0 0 0 1 0 0 0 0]

酒店 = [0 0 0 0 0 0 0 1 0 0 0 0 0 0 0]

向量维度 = 词汇表中的词数（例如，500,000+）

将词视为离散符号的问题

示例： 在网页搜索中，如果用户搜索 “西雅图汽车旅馆”， 我们希望匹配包含 “西雅图酒店”的文档

汽车旅馆 = [0 0 0 0 0 0 0 0 0 0 1 0 0 0 0]
酒店 = [0 0 0 0 0 0 0 1 0 0 0 0 0 0 0]

但这两个向量是 正交的

对于一热向量没有自然的 相似性 概念！

解决方案：

- 可以尝试依赖WordNet的同义词列表来获取相似性吗？
  - 但众所周知，它的效果很差：不完整性等。
- **相反：学习在向量中编码相似性**

通过上下文表示单词



- **分布语义学：一个词的意义由经常出现在其附近的词所决定**
  - *“你可以通过一个词所处的环境来了解它”*（J. R. Firth 1957: 11）
  - 现代统计自然语言处理最成功的理念之一！
- 当一个词 *w* 出现在文本中时，它的 上下文 是出现在附近的词的集合（在一个固定大小的窗口内）。
- 我们使用 *w* 的多种上下文来构建 *w 的表示*

… 政府债务问题转变为银行危机，如2009年所发生的…

… 说欧洲需要统一的银行监管来取代杂乱无章的现状…

… 印度刚刚给其银行系统注入了一剂强心针…

这些 上下文词 将代表 *银行*

词向量

我们将为每个单词构建一个密集向量，选择使其与出现在相似上下文中的单词的向量相似，测量相似性的方法是向量的 点 （标量） 积

$$银行 = \begin{pmatrix} 0.286 \\ 0.792 \\ -0.177 \\ -0.107 \\ 0.109 \\ -0.542 \\ 0.349 \\ 0.271 \end{pmatrix} \qquad 货币 = \begin{pmatrix} 0.413 \\ 0.582 \\ -0.007 \\ 0.247 \\ 0.216 \\ -0.718 \\ 0.147 \\ 0.051 \end{pmatrix}$$

注意： 词向量 也称为 （词）嵌入 或 （神经）词表示
它们是 分布式 表示

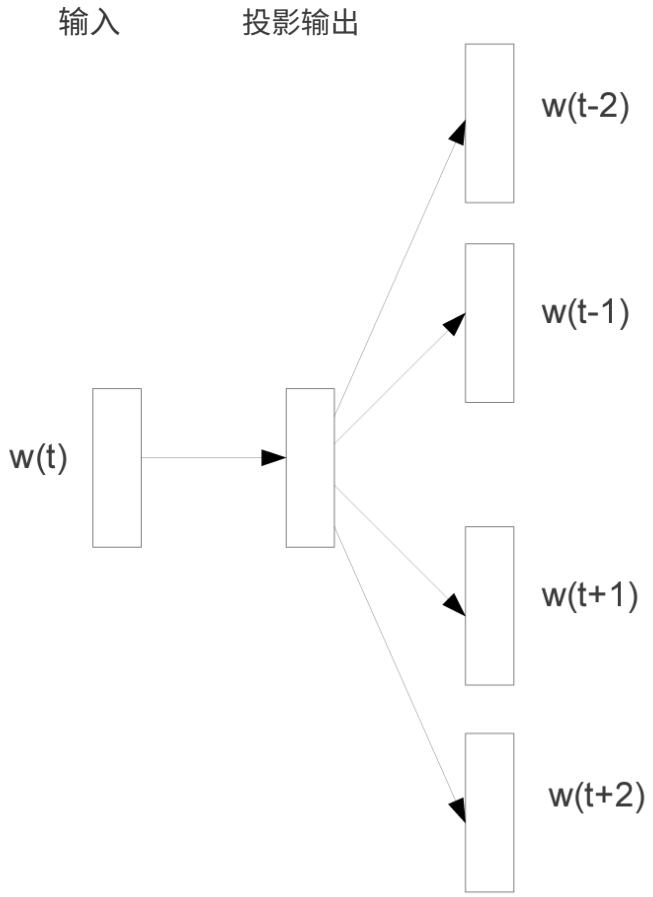
3. Word2vec: 概述

Word2vec 是一个学习词向量的框架

（Mikolov 等，2013）

思想：

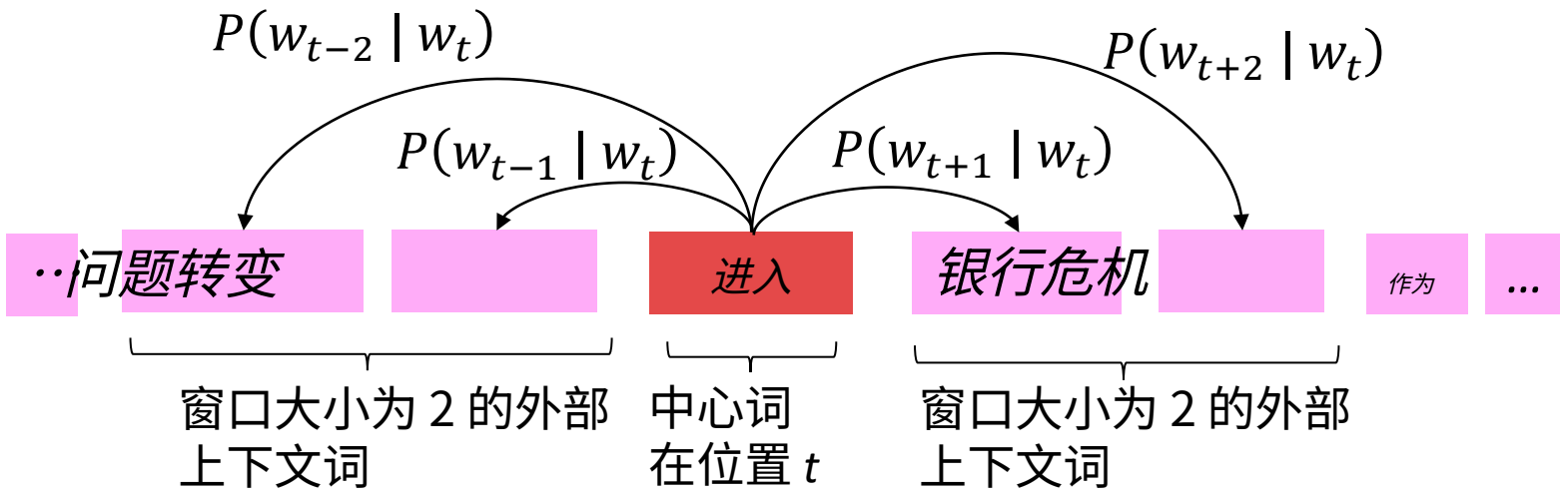
- 我们有一个大型语料库（“主体”）的文本：一长串单词
- 固定词汇表中的每个单词都由一个 向量表示
- 遍历文本中的每个位置t，中心词为单词c，上下文（“外部”）词为o
- 使用 单词向量c 和o 的相似性来 计算 给定 c *的* o *的概率*（或反之）
- 不断调整词向量 以最大化这个概率



跳字模型
(米科洛夫等，2013)

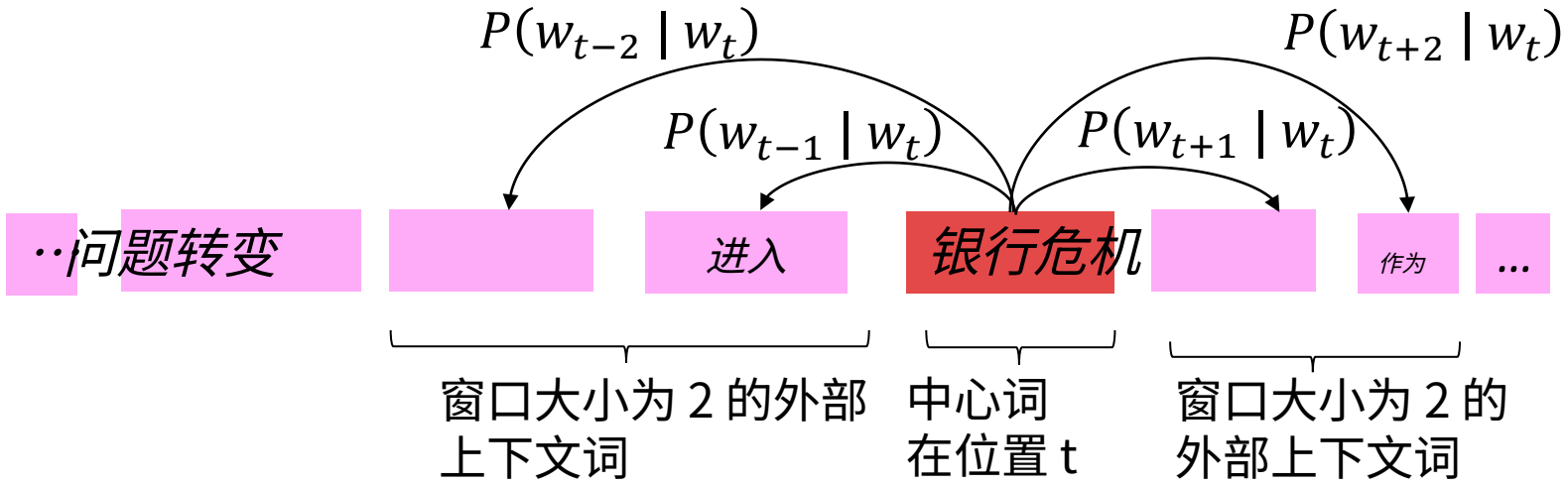
Word2Vec 概述

计算 $P(w_{t+j} \mid w_t)$ 的示例窗口和过程 （ ）



Word2Vec 概述

计算 $P(w_{t+j} \mid w_t)$ 的示例窗口和过程



Word2Vec: 目标函数

对于每个位置 $t = 1, \ldots, T$ 预测固定大小窗口 m 内的上下文词，给定中心词 w_t。 数据似然性：

似然 =

$$L(\theta) = \prod_{t=1}^{T} \prod_{\substack{-m \le j \le m \\ j \neq 0}} P(w_{t+j} \mid w_t; \theta)$$

θ 是所有变量需要优化

有时称为 *成本* 或 *损失* 函数

目标函数 $J(\theta)$ 是 (平均) 负对数似然性：

$$J(\theta) = -\frac{1}{T} \log L(\theta) = -\frac{1}{T} \sum_{t=1}^{T} \sum_{\substack{-m \le j \le m \\ j \neq 0}} \log P(w_{t+j} \mid w_t; \theta)$$

最小化目标函数 $\Leftrightarrow$ 最大化预测准确性

Word2Vec: 目标函数

- 我们想要最小化目标函数：

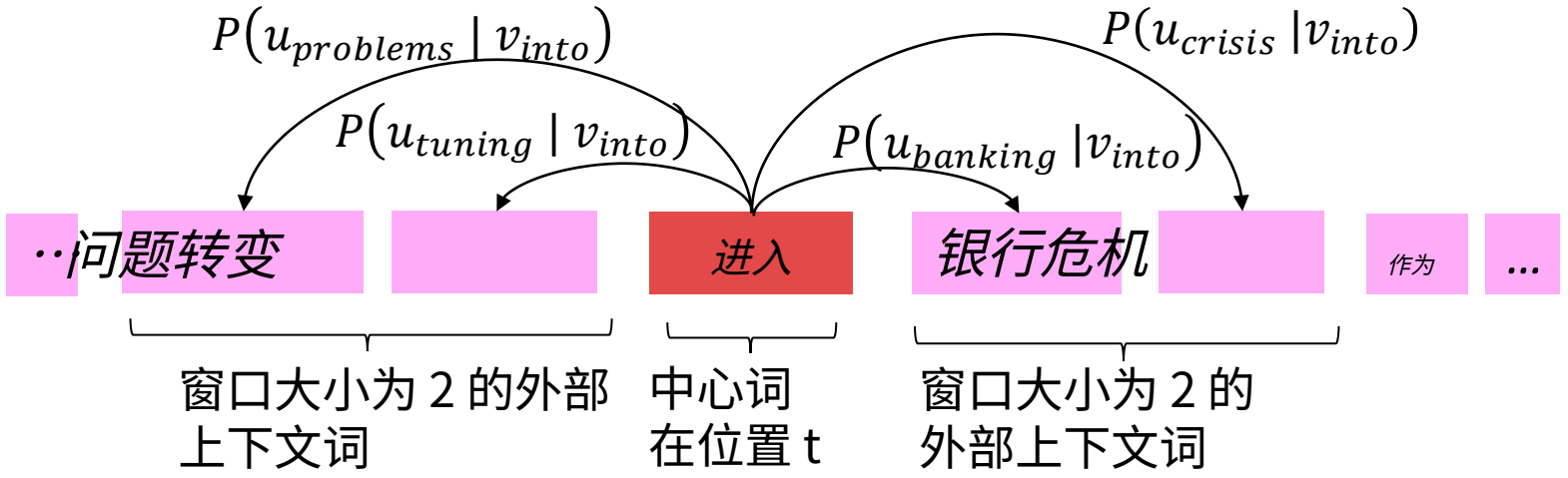
$$J(\theta) = -\frac{1}{T}\sum_{t=1}^{T}\sum_{\substack{-m\le j\le m\\ j\neq 0}} \log P\left(w_{t+j} \mid w_t; \theta\right)$$

- 问题：如何计算 $P\left(w_{t+j} \mid w_t; \theta\right)$？
- 答案：我们将为每个单词 w 使用两个 向量：
  - 当 w 是中心词时，使用 v_w
  - 当 w 是上下文词时，使用 u_w
- 然后对于中心词 c 和上下文词 o：

$$P(o|c) = \frac{\exp(u_o^T v_c)}{\sum_{w\in V}\exp(u_w^T v_c)}$$

Word2Vec与向量

- 计算 $P(w_{t+j} \mid w_t)$ 的示例窗口和过程
- $P(u_{problems} \mid v_{into})$ 缩写为 $P(problems \mid into; \theta)$



Word2Vec：预测函数

② 指数运算使任何数变为正数

$$P(o|c) = \frac{\exp(u_o^T v_c)}{\sum_{w \in V} \exp(u_w^T v_c)}$$

① 点积比较 o 和 c 的相似性。
$u^T v = u \cdot v = \sum_{i=1}^{n} u_i v_i$
更大的点积 = 更大的概率

③ 在整个词汇表上进行归一化
以给出概率分布

- 这是一个 softmax 函数 $\mathbb{R}^n \to (0,1)^n$ 的示例

打开区域

$$\text{softmax}(x_i) = \frac{\exp(x_i)}{\sum_{j=1}^{n} \exp(x_j)} = p_i$$

- softmax 函数将任意值 x_i 映射到概率分布 p_i
  - “max” 因为放大了最大 x_i 的概率
  - “soft”因为仍然给较小的 x_i 分配了一定的概率
  - 在深度学习中经常使用

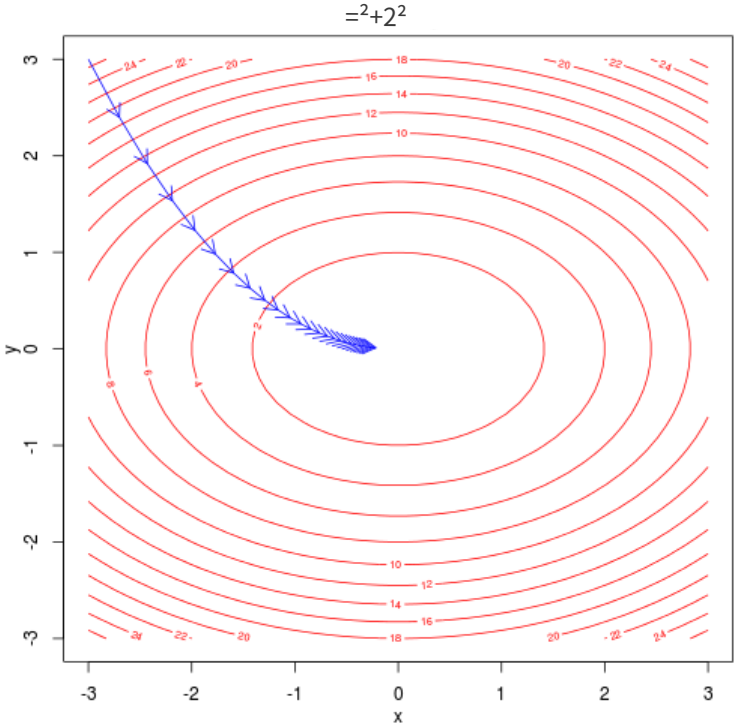
但这个名字有点奇怪
因为它返回的是一个分布！

训练模型：优化参数值以最小化损失

为了训练模型，我们逐步调整参数以最小化损失

- 回忆：θ 表示 所有 的模型参数，形成一个长向量
- 在我们的情况下，具有 维向量和 V个单词，我们有→
- 记住：每个单词都有两个向量

$$\theta = \begin{bmatrix} \text{瓦尔德瓦克} \\ \text{《Va的目标语言译文》} \\ \vdots \\ \text{《Vzebra的目标语言译文》} \\ \text{《Uaardvark的目标语言译文》} \\ \text{《Ua的目标语言译文》} \\ \vdots \\ \text{《Uzebra的目标语言译文》} \end{bmatrix} \quad \text{《} \in R^{2dv} \text{的目标语言译文》}$$



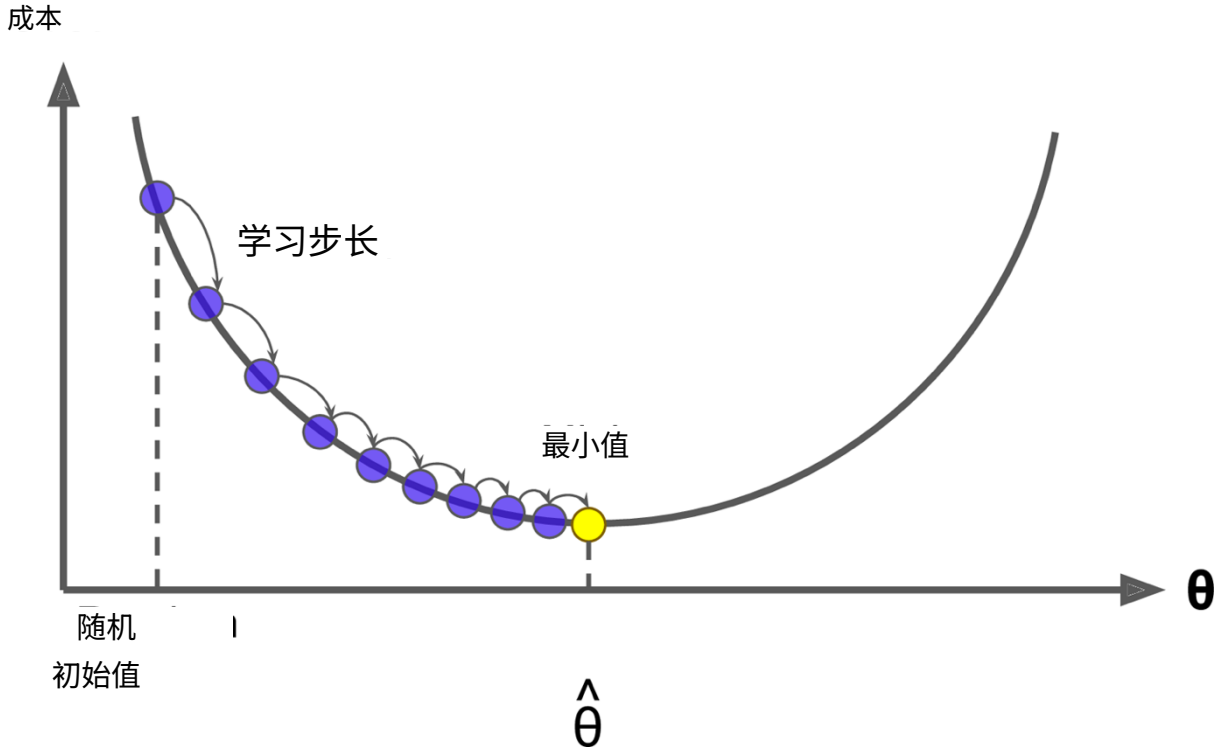
- 我们通过沿着梯度下降来优化这些参数（见右图）
- 我们计算 所有 向量的梯度！

互动环节！

- $L(\theta) = \prod_{t=1}^{T} \prod_{\substack{-m \le j \le m \\ j \neq 0}} P(w_{t+j} \mid w_t; \theta)$
- 对于中心词 c 和上下文词 o: $P(o \mid c) = \frac{\exp(u_o^T v_c)}{\sum_{w \in V} \exp(u_w^T v_c)}$

4. 优化：梯度下降

- 我们有一个成本函数 $J(\theta)$ 我们想要最小化
- 梯度下降是一个最小化 的算法 （ ）
- 想法：对于当前的θ值，计算 θ的梯度，然后在负梯度方向上迈出小步
  重复。



注意：我们的
目标
可能不是
像这样是凸的
但生活结

果是还不错
☺

梯度下降

- 更新方程（矩阵表示法）：

$$\theta^{new} = \theta^{old} - \alpha \nabla_{\theta} J(\theta)$$

α= *步长* 或 *学习率*

- 更新方程（单个参数）：

$$\theta_j^{new} = \theta_j^{old} - \alpha \frac{\partial}{\partial \theta_j^{old}} J(\theta)$$

- 算法：

```
while True:
    theta_grad = evaluate_gradient(J,corpus,theta)
    theta = theta - alpha * theta_grad
```

随机梯度下降

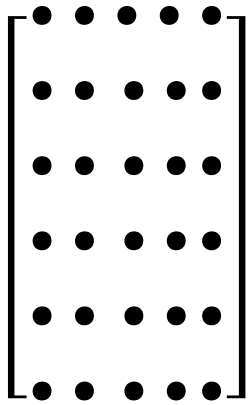
- 问题: $J(\theta)$ 是语料库中 所有 窗口的函数（可能有数十亿个！）
  - 所以 $\nabla_\theta J(\theta)$ 是 计算成本非常高
- 你会等很长时间才进行一次更新！

- 对于几乎所有神经网络来说，这个主意非常糟糕！
- **解决方案：随机梯度下降（SGD）**
  - 重复采样窗口，并在每次之后进行更新
- 算法：

小批量梯度下降

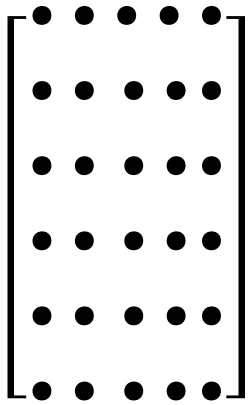
```
while True:
    窗口 = sample_window (语料库)
    theta_grad = evaluate_gradient (J,窗口, theta)
    theta = theta - alpha * theta_grad
```

Word2vec参数 … 和计算



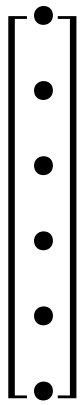
U

外部



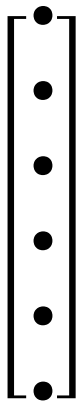
V

中心



$U \cdot v_4{}^T$

点积



$\text{softmax}(U \cdot v_4{}^T)$

概率

“词袋”模型！

模型在每个位置做出相同的预测

我们希望模型对出现在
上下文中的所有词给出合理高的
概率估计（尽可能多）

Word2vec通过将相似的词放置在空间中相邻来最大化目标函数



Word2vec 算法家族（Mikolov 等，2013）：更多细节

为什么使用两个向量？→ 更容易优化。最后取平均

- 但可以仅用每个单词一个向量来实现算法……这有一点帮助

两种模型变体：

1. 跳字模型（SG）
   给定中心词，预测上下文（“外部”）词（位置无关）
2. 连续词袋模型（CBOW）
   从上下文词（词袋）中预测中心词

我们展示了： 跳字模型

训练的损失函数：

1. 简单的 softmax（简单但代价高的损失函数，当输出类别很多时）
2. 更优化的变体，如层次化软最大值
3. 负采样

到目前为止，我们解释了 天真软最大值

带有负采样的跳字模型

- 归一化项在计算上是昂贵的（当输出类别很多时）：

- $$P(o|c) = \frac{\exp(u_o^T v_c)}{\sum_{w \in V} \exp(u_w^T v_c)}$$

  对词的一个大求和

- 因此，标准的word2vec实现了带有 负采样的跳字模型

- 主要思想：训练二元逻辑回归以区分真实对（中心词和其上下文窗口中的一个词）与几个“噪声”对（中心词与一个随机词配对）

带有负采样的跳字模型(Mikolov et al. 2013)

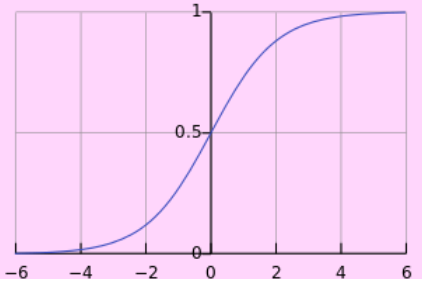
- 我们取k 负样本（使用词概率）
- 最大化真实外部词出现的概率；
  最小化随机词在中心词周围出现的概率
- 使用与本课程一致的符号，我们最小化：

$$J_{neg-sample}(\boldsymbol{u}_o, \boldsymbol{v}_c, U) = -\log\sigma(\boldsymbol{u}_o^T\boldsymbol{v}_c) - \sum_{k\in\{K\ sampled\ indices\}} \log\sigma(-\boldsymbol{u}_k^T\boldsymbol{v}_c)$$

sigmoid 而不是 softmax

- 逻辑斯蒂/ sigmoid 函数：
  (我们很快就会成为好朋友☺)

$$\sigma(x) = \frac{1}{1+e^{-x}}$$



- 样本 P(w)=U(w)$^{3/4}$/Z，单元分布 U(w) 提升到 3/4 次方
  - 这个幂使得不常见的词被更频繁地抽样

带有负采样的随机梯度 [旁注]

- 我们在每个窗口中迭代地获取梯度以进行随机梯度下降（SGD）
- 在每个窗口中，我们最多只有 $2m+1$ 个词加上 $2km$ 个负样本词，因此 $\nabla_\theta J_t(\theta)$ 是非常稀疏的！

$$\nabla_\theta J_t(\theta) = \begin{bmatrix} 0 \\ \vdots \\ \nabla_{v_{like}} \\ \vdots \\ 0 \\ \vdots \\ \blacktriangledown U\text{学习} \\ \vdots \end{bmatrix}$$

《∈ R2dv的目标语言译文》

✓ 用户界面

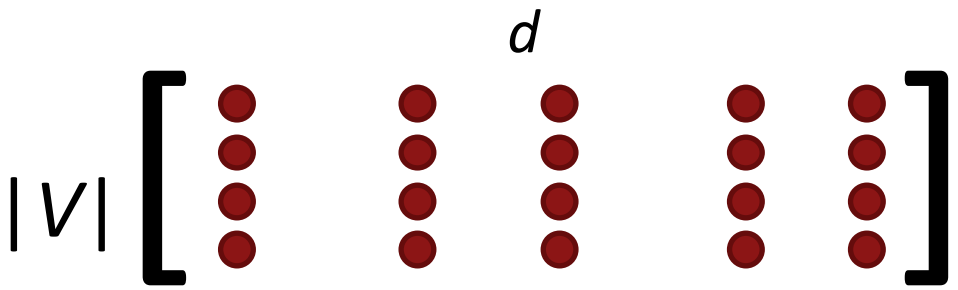
带有负采样的随机梯度 [旁注]

- 我们可能只更新实际出现的词向量！

- 解决方案：要么你需要稀疏矩阵更新操作，只更新完整嵌入矩阵 U 和 V的某些 行，要么你需要保留一个词向量的哈希表

行而非列

在实际的深度学习包中！



- 如果你有数百万个词向量并进行分布式计算，重要的是不要发送巨大的更新！

5. 为什么不直接捕捉共现计数?

遍历整个语料库（可能很多次）有些奇怪;
我们为什么不直接累积所有统计数据，看看哪些词出现在彼此附近呢？！？

构建共现矩阵 X

- 2个选项：窗口 vs. 整个文档
- 窗口：类似于word2vec，在每个词周围使用窗口 → 捕捉一些句法和语义信息（“词空间”）
- 词文档共现矩阵将提供一般主题（所有体育术语将有类似的条目）导致“潜在语义分析”（“文档空间”）

示例：基于窗口的共现矩阵

- 窗口长度 1（更常见：5–10）
- 对称（左或右上下文无关紧要）

- 示例语料库：
  - 我喜欢深度学习
  - 我喜欢自然语言处理
  - 我喜欢飞行

计数	我	喜欢	享受	深入	学习	NLP	飞行	.
我	0	2	1	0	0	0	0	0
喜欢	2	0	0	1	0	1	0	0
享受	1	0	0	0	0	0	1	0
深入	0	1	0	0	1	0	0	0
学习	0	0	0	1	0	0	0	1
NLP	0	1	0	0	0	0	0	1
飞行	0	0	1	0	0	0	0	1
.	0	0	0	0	1	1	1	0

共现向量

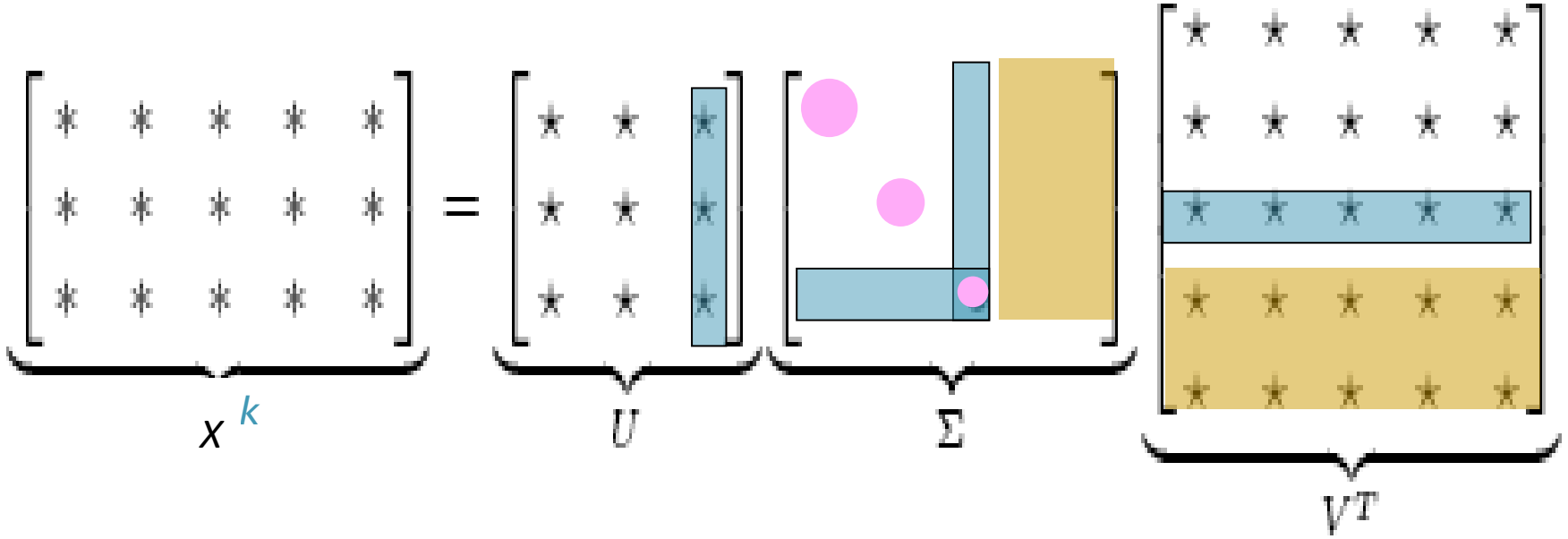
- 简单计数共现向量
  - 向量随着词汇量的增加而增大
  - 维度非常高：需要大量存储（尽管是稀疏的）
  - 后续分类模型存在稀疏性问题→ 模型的鲁棒性较差

- 低维向量
  - 想法：在固定的小数量维度中存储“最”重要的信息：一个密集向量
  - 通常为25–1000维，类似于word2vec
  - 如何降低维度?

经典方法：对 X 进行降维 (HW1)

共现矩阵的奇异值分解 X

将 X 分解为 $U\Sigma V^T$，其中 U 和 V 是正交归一的（单位向量和正交）



仅保留 k 个奇异值，以便进行泛化。

$\hat{X}$ 是 X 的最佳秩 k 近似，基于最小二乘法。

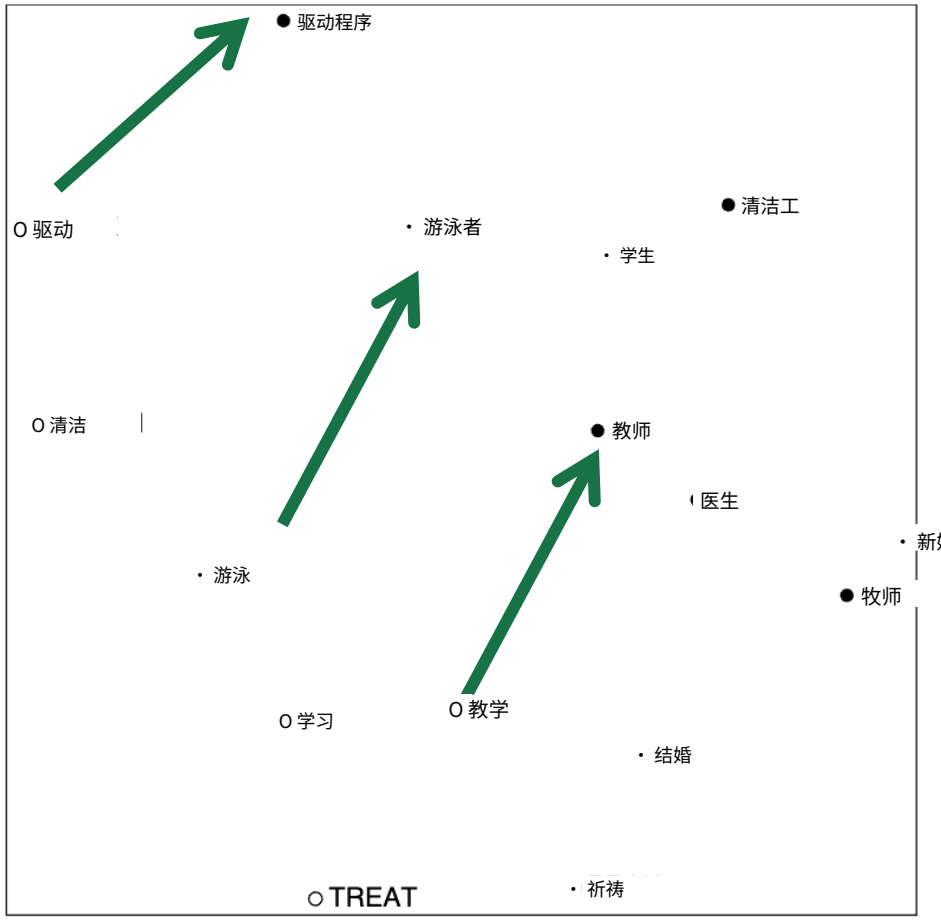
经典线性代数结果。对于大矩阵计算成本高。

对 X 的黑客

- 在原始计数上运行 SVD 效果不好！！！

- 在单元格中缩放计数可以帮助*很多*
  - 问题：功能词（*the, he, has*）过于频繁→ 语法影响过大。
    一些修复方法：
    - 记录频率
    - min(X, t)，其中$t \approx 100$
    - 忽略功能词
- 倾斜窗口，更加重视靠近的词而非远离的词
- 使用相关性而非计数，然后将负值设为0
- 等等。

在缩放向量中出现有趣的语义模式



来自COALS模型
Rohde等人，2005年。基于词汇共现的改进语义相似性模型

GloVe [Pennington, Socher, and Manning, EMNLP 2014]:
在向量差异中编码意义成分

问：我们如何在词向量空间中捕捉共现概率的比率作为线性意义成分？

GloVe [Pennington, Socher, and Manning, EMNLP 2014]: 在向量差异中编码意义成分

问：我们如何在词向量空间中捕捉共现概率的比率作为线性意义成分？

A: 对数双线性模型：

$$w_i \cdot w_j = \log P(i|j)$$

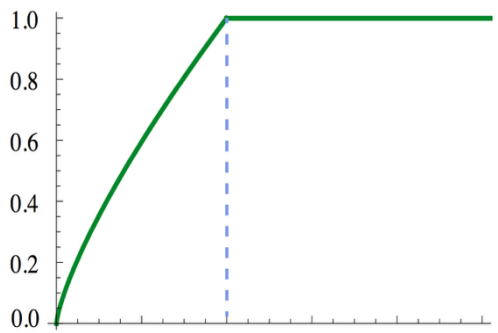
具有向量差异

$$w_x \cdot (w_a - w_b) = \log \frac{P(x|a)}{P(x|b)}$$

损失：

$$J = \sum_{i,j=1}^{V} f\left(X_{ij}\right)\left(w_i^T \tilde{w}_j + b_i + \tilde{b}_j - \log X_{ij}\right)^2$$

$f \sim$



- 快速训练
- 可扩展到大型语料库

6. 如何评估词向量?

- 与自然语言处理中的一般评估相关：内在评估与外在评估
- 内在的：
  - 对特定/中间子任务的评估
  - 计算速度快
  - 有助于理解该系统
  - 除非与真实任务建立相关性，否则不清楚是否有帮助
- 外在的：
  - 对真实任务的评估
  - 计算准确性可能需要很长时间
  - 不清楚是子系统的问题还是其交互或其他子系统的问题
  - 如果用另一个子系统完全替换一个子系统能提高准确性 → 胜利!

《内在词向量评估》

- 《词向量类比》

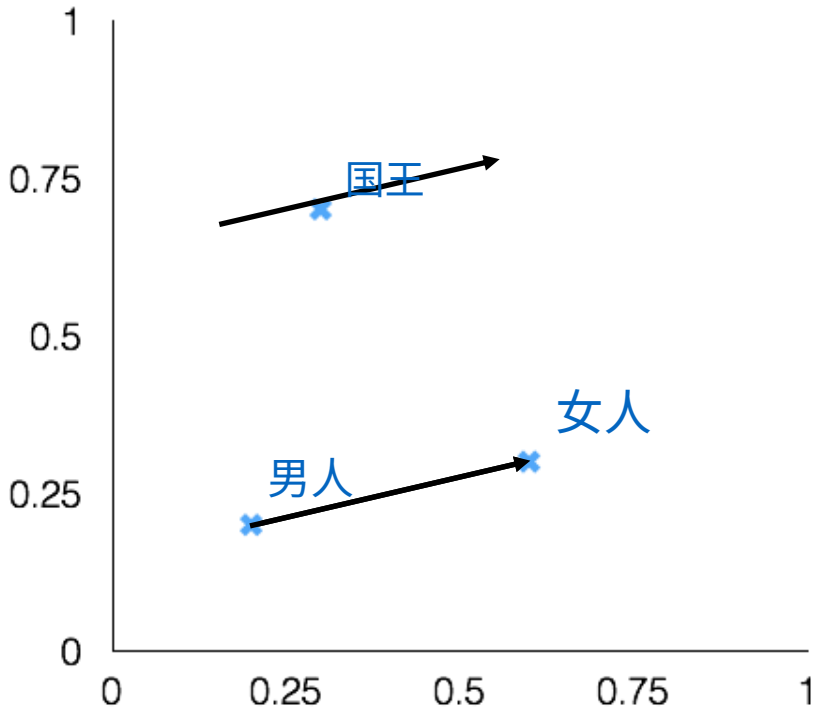
《a:b:: c:?》

男人:女人 :: 国王:?

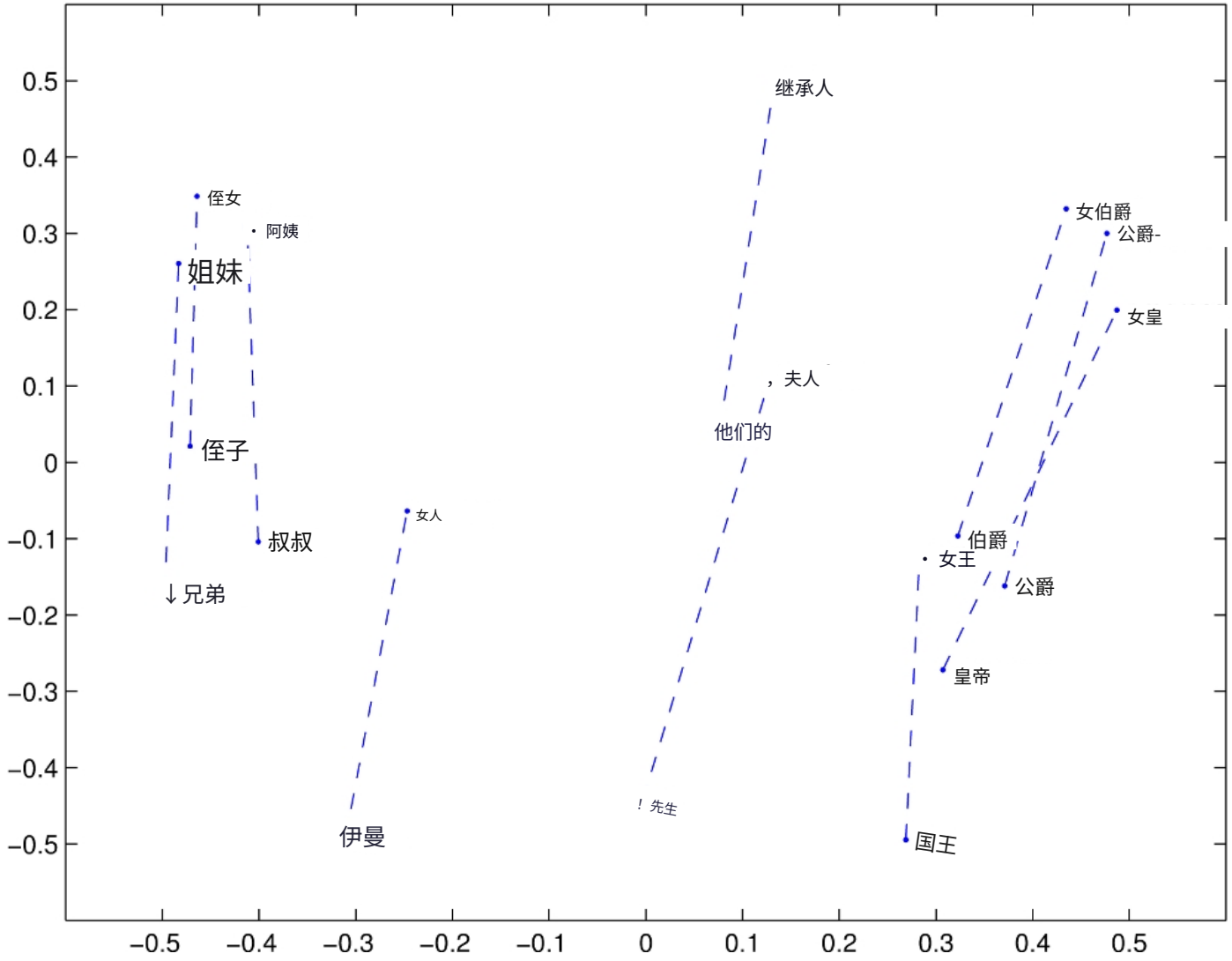
$$d = \arg\max_i \frac{(x_b - (x_a + x_c)) T x_i}{\|x_b - x_a + x_c\|}$$

- 通过评估词向量的余弦距离在加法后的表现，来捕捉直观的语义和句法类比问题
- 丢弃输入词汇进行搜索(!)
- 问题：如果信息存在但

不是线性的怎么办?



GloVe 可视化


0.5
0.4
0.3
0.2
0.1
0
−0.1
−0.2
−0.3
−0.4
−0.5
−0.5
−0.4
−0.3
−0.2
−0.1
0
0.1
0.2
0.3
0.4
0.5
侄女
阿姨
姐妹
侄子
叔叔
↓兄弟
女人
伊曼
继承人
，夫人
他们的
！先生
女伯爵
公爵-
女皇
伯爵
女王
公爵
皇帝
国王

意义相似性：另一种内在词向量评估

- 词向量距离及其与人类判断的相关性
- 示例数据集：WordSim353
  http://www.cs.technion.ac.il/~gabr/resources/data/wordsim353/

词 1	词 2	人类（平均）
老虎	猫	7.35
老虎	老虎	10
书	纸	7.46
计算机	互联网	7.58
飞机	汽车	5.77
教授	医生	6.62
股票	电话	1.62
股票	CD	1.31
股票	美洲虎	0.92

相关性评估

- 词向量距离及其与人类判断的相关性

模型	大小	WS353	MC	RG	SCWS	RW
SVD	6B	35.3	35.1	42.5	38.3	25.6
SVD-S	6B	56.5	71.5	71.0	53.6	34.7
SVD-L	6B	65.7	72.7	75.1	56.5	37.0
CBOW†	6B	57.2	65.6	68.2	57.0	32.5
SG†	6B	62.8	65.2	69.7	58.1	37.2
GloVe	6B	65.8	72.7	77.8	53.9	38.1
SVD-L	42B	74.0	76.4	74.1	58.3	39.9
GloVe	42B	**75.9**	**83.6**	**82.9**	**59.6**	**47.8**
CBOW*	100B	68.4	79.6	75.4	59.4	45.5

外部词向量评估

- 一个好的词向量应该直接帮助的例子： 命名实体识别：识别对人、组织或地点的引用：

  克里斯·曼宁 住在 帕洛阿尔托。

模型	开发测试		ACE	MUC7
离散 91.0 85.4			77.4	73.4
SVD	90.8	85.7	77.3	73.7
SVD-S	91.0	85.5	77.6	74.3
SVD-L	90.5	84.8	73.6	71.5
HPCA	92.6	**88.7**	81.7	80.7
HSMN	90.5	85.7	78.7	74.7
CW	92.2	87.4	81.7	80.2
CBOW	93.1	88.2	82.2	81.1
GloVe	**93.2**	88.3	**82.9**	**82.2**

讲座计划

讲座 2：词向量

1. 课程组织（3分钟）
2. Word2vec 介绍（15分钟）
3. Word2vec 目标函数梯度 (25 分钟)
4. 优化基础（5分钟）
5. 我们能通过计数更有效地捕捉词义的本质吗？ (10m)
6. 评估词向量（10分钟）

关键目标：理解词义可以用高维向量表示
实数，并且在课程结束时能够阅读词嵌入论文